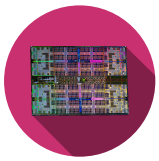
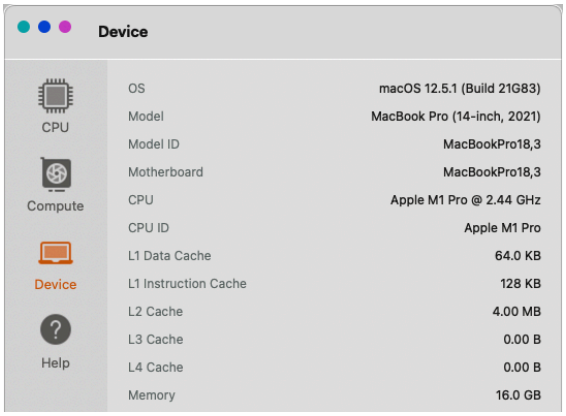


Benchmark

Lab Computer Architecture

Contents

Table 1 -



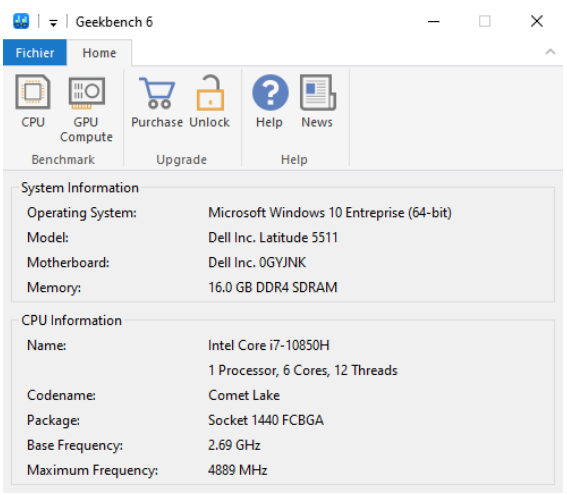


Table 2 -

Person	Operating System (OS)	Central Processing Unit (CPU)	CPU -cores	CPU Arch.	Frequency [GHz]
Silvan Zahno	macOS 12.5.1	Apple M1 Pro	10	AARCH64	2.44
Silvan Zahno	macOS 15.5	Apple M4 Max	16(12p/4e)	AARCH64	4.49
Axel Amand	Win 11 10.0.22	I5-12600K	10	AMD64	3.7

L1 Data [kB]	L1 Instruction [kB]	L2 [MB]	L3 [MB]	L4 [MB]	RAM [GB]
64	128	4	n.a.	n.a.	16
64	128	4	n.a.	n.a.	64
48*8	32*8	1.25*2	20	0	32

Table 3 -

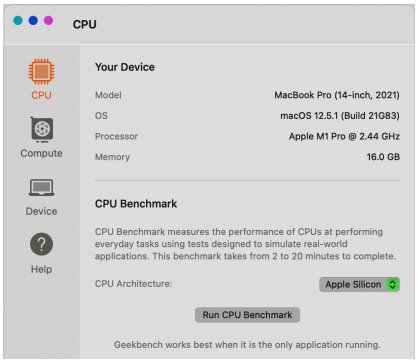


Figure 1 -

Person	**Single-Core Score**	**Multi-Core Score**
Silvan Zahno (M1)	2279	11839
Silvan Zahno (M4)	3813	25272
Axel Amand	2372	12566

Table 4 -

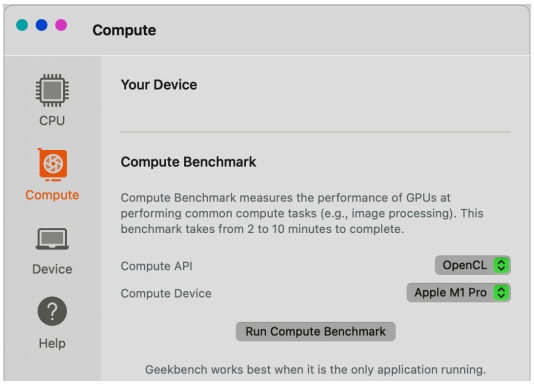


Figure 2 -

Person	**OpenCL**	**Metal**	**CUDA**	**Vulkan**
Silvan Zahno (M1)	40823	67476	n.a.	n.a.
Silvan Zahno (M4)	111779	166950	n.a.	n.a.
Axel Amand (UHD Graphics 770)	7516	n.a.	n.a.	8502
Axel Amand (GTX1080)	48083	n.a.	51896	65467

Table 5 -

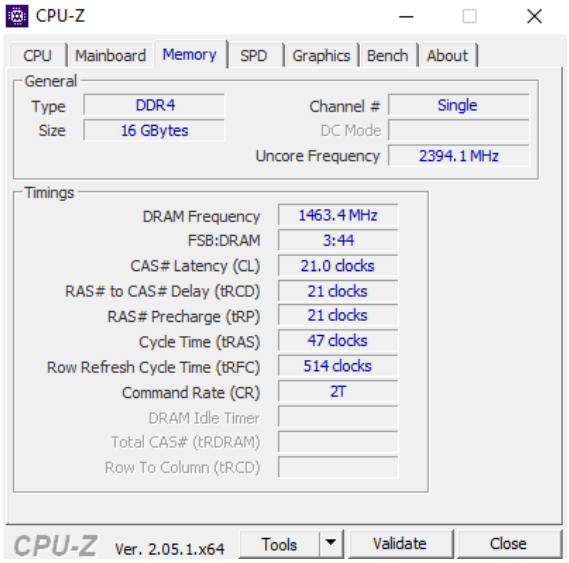


Figure 3 -

0.0.1 MAC

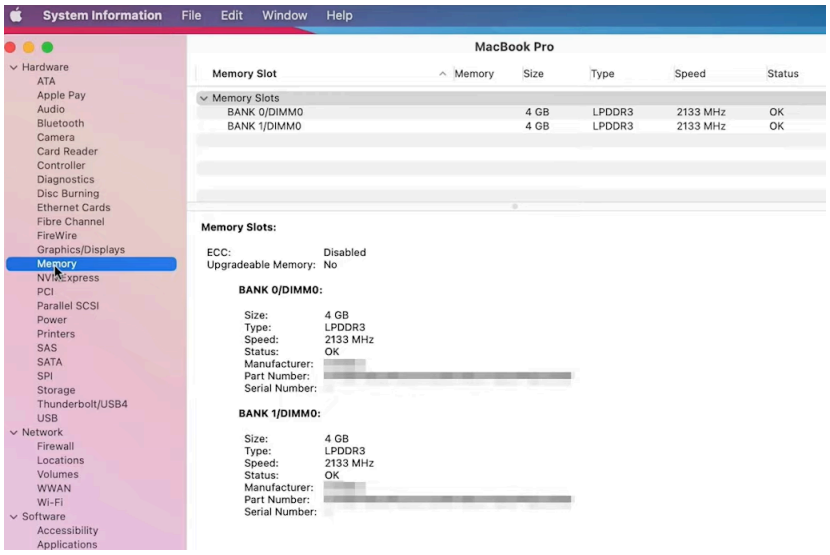
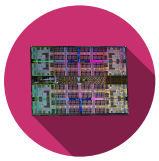


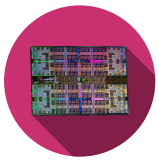
Figure 4 -

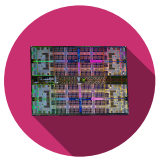


```
# goto the corresponding folder (command to be adapted)
cd car-labs/bem/zip

# Linux and MacOS
chmod +x zip-benchmarking.bash
./zip-benchmarking.bash

# Windows
.\zip-benchmarking.bat
```

Person	**Unzip**	**Zip**
Silvan Zahno (M1)	2.651	3.619
Silvan Zahno (M4)	1.717	2.383
Axel Amand	1.66	2.97

Table 6 -





```
procedure bubbleSort(A : list of sortable items)
  n := length(A)
  repeat
    swapped := false
    for i := 1 to n-1 inclusive do
      { if this pair is out of order }
      if A[i-1] > A[i] then
        { swap them and remember something changed }
        swap(A[i-1], A[i])
        swapped := true
      end if
    end for
  until not swapped
end procedure
```

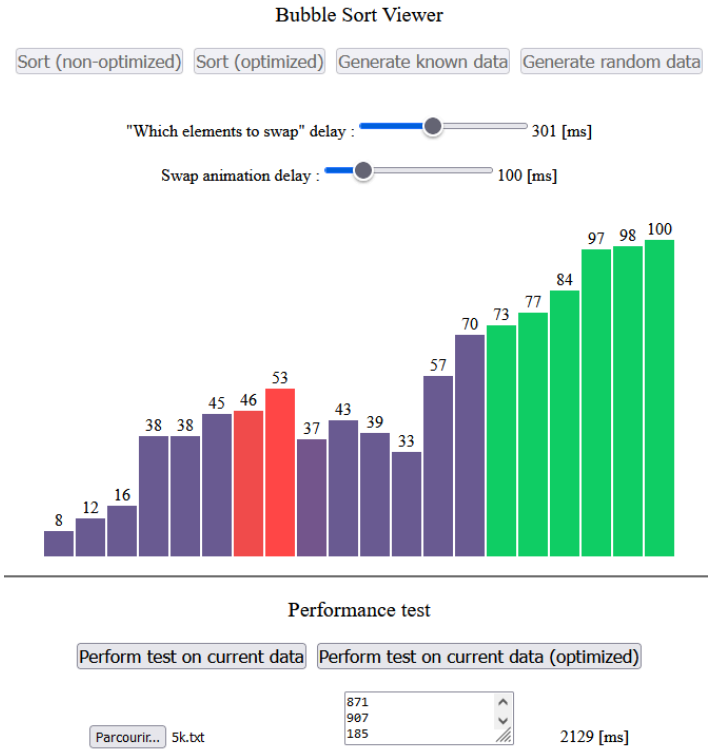
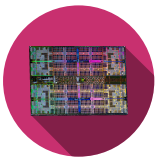


Figure 5 - Bubblesort Javascript

```
Select data file on the newly opened GUI
Reading data into memory
Sorting
 * Bubble sort (non-opti) done in 189 [ms]
Sorting
 * Bubble sort (optimized) done in 156 [ms]
```

Figure 6 - Bubblesort Rust



```
Bubble Sort - C version
    Data Selection

1. 20
2. 1k
3. 5k
4. 10k
5. 50k
6. 100k
0. Exit

Enter your choice: 6
Checking args
Reading data into memory
 * Found 100000 data
Copying data for opti. algo
Sorting
 * Bubble sort (non-opti) done in 24883 [ms]
 * Bubble sort (optimized) done in 23289 [ms]
```

Figure 7 - Bubblesort C

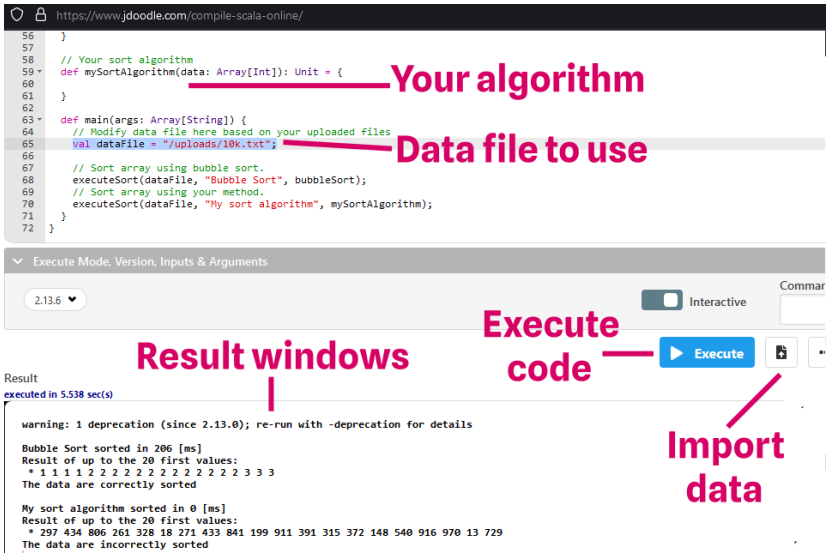
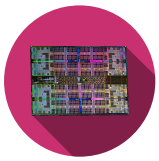


Figure 8 -

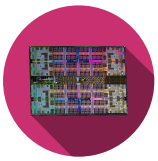
0.0.2 Todo

0.0.3 Template scala

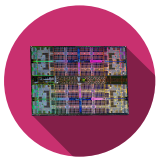
```scala
object JDoodle {

  // Reads file and create memory array
  def readFile(filename: String): Array[Int] = {
    val bufferedSource = io.Source.fromFile(filename)
    val lines = (for (line <- bufferedSource.getLines()) yield line.toInt).toArray
    bufferedSource.close
    lines
  }

  // Execute the given sort function with timings check
  def executeSort(filename: String, sortName: String, func: (Array[Int]) => Unit): Unit
```



```
= {
    var data = readFile(filename);
    var start = System.currentTimeMillis();
    func(data);
    var end = System.currentTimeMillis();

    printf("\n%s sorted in %d [ms]\nResult of up to the 20 first values:\n * ",
sortName, end-start);
    var i = 0;
    while(i < 20 && i < data.length){
      printf("%d ", data(i));
      i += 1;
    }
    printf("\nThe data are %s sorted\n", if(checkValues(data)) "correctly" else
"incorrectly");
  }

  // Test if the array is correctly sorted
  def checkValues(data: Array[Int]): Boolean = {
    var last = data(0);
    var dta = 0;
    for(dta <- data){
        if (dta < last){return false;}
        last = dta;
    }
    return true;
  }

  // A bubble sort example
  def bubbleSort(data: Array[Int]): Unit = {
    var i: Int = 0
    var j: Int = 0
    var t: Int = 0

    while (i < data.length) {
      j = data.length - 1;
      while (j > i) {
        if (data(j) < data(j - 1)) {
          t = data(j);
          data(j) = data(j - 1);
          data(j - 1) = t;
        }
        j = j - 1
      }
      i = i + 1
    }
  }

  // Your sort algorithm
  def mySortAlgorithm(data: Array[Int]): Unit = {

  }

  def main(args: Array[String]) {
    // Modify data file here based on your uploaded files
    val dataFile = "/uploads/10k.txt";

    // Sort array using bubble sort.
```



```
    executeSort(dataFile, "Bubble Sort", bubbleSort);
    // Sort array using your method.
    executeSort(dataFile, "My sort algorithm", mySortAlgorithm);
  }
}
```



Glossary

CPU – **Central Processing Unit** 2

OS – **Operating System** 2